URL, it could elect to receive the results in a code-friendly format (YAML, JSON, or XML, perhaps).

We've already seen how Rails can use the HTTP Accept header in a `respond_to` block in the controller. However, it isn't always easy (and sometimes it's plain impossible) to set the Accept header. To deal with this, Rails allows you to pass the format of response you'd like as part of the URL. As you've seen, Rails accomplishes this by including a field called `:format` in your route definitions. To do this, set a `:format` parameter in your routes to the file extension of the MIME type you'd like returned:

```
GET    /products(.:format)
       {:action=>"index", :controller=>"products"}
```

Because a full stop (period) is a separator character in route definitions, `:format` is treated as just another field. Because we give it a `nil` default value, it's an optional field.

Having done this, we can use a `respond_to` block in our controllers to select our response type depending on the requested format:

```
def show
  respond_to do |format|
    format.html
    format.json { render json: @product.to_json }
  end
end
```

Given this, a request to `/store/show/1` or `/store/show/1.html` will return HTML content, while `/store/show/1.xml` will return XML, and `/store/show/1.json` will return JSON. You can also pass the format in as an HTTP request parameter:

```
GET HTTP://pragprog.com/store/show/123?format=xml
```

Although the idea of having a single controller that responds with different content types seems appealing, the reality is tricky. In particular, it turns out that error handling can be tough. Although it's acceptable on error to redirect